

COL774 Assignment 2

Shrenik Sakala (2022MT11920)

October 8, 2025

Text Classification using Naive Bayes

Model comparison using only content as feature

Model	Accuracy	Precision	Recall	F1-score
Unigrams (No pre-processing)	0.97471	0.97487	0.97471	0.97462
Unigrams (Stopword removal only)	0.97304	0.97318	0.97304	0.97291
Unigrams (Stemming only)	0.97010	0.97031	0.97010	0.96997
Unigrams (Stopword removal + stemming)	0.96634	0.96657	0.96634	0.96613
Unigrams + Bigrams (No pre-processing)	0.99001	0.99002	0.99001	0.99000
Unigrams + Bigrams (Stopword removal only)	0.99262	0.99263	0.99262	0.99261
Unigrams + Bigrams (Stemming only)	0.98863	0.98867	0.98863	0.98861
Unigrams + Bigrams (Stopword removal + stemming)	0.99031	0.99034	0.99031	0.99029

Table 1: Comparison on Train Data

Model	Accuracy	Precision	Recall	F1-score
Unigrams (No pre-processing)	0.95811	0.95846	0.95811	0.95792
Unigrams (Stopword removal only)	0.95500	0.95524	0.95500	0.95469
Unigrams (Stemming only)	0.95363	0.95405	0.95363	0.95341
Unigrams (Stopword removal + stemming)	0.94817	0.94846	0.94817	0.94773
Unigrams + Bigrams (No pre-processing)	0.96737	0.96766	0.96737	0.96727
Unigrams + Bigrams (Stopword removal only)	0.96360	0.96379	0.96360	0.96338
Unigrams + Bigrams (Stemming only)	0.96551	0.96582	0.96551	0.96539
Unigrams + Bigrams (Stopword removal + stemming)	0.95963	0.95992	0.95963	0.95933

Table 2: Comparison on Test Data

We observe that when we use only content as a feature without any pre-processing we get better accuracies than after removing stop words or using stemming. An explanation for the same could be the **loss of context** that some of the stop words are adding to the document. Similarly stemming also leads to loss of contextual information for example tenses. This contextual information might be vital for this particular classification task and hence loss of the same leads to poorer results. Including bigrams as features increases accuracy as they help us **capture interactions of consecutive words** along with the individual words themselves. This helps us capture more meaning about the context in which the words are being used and hence helps reduce misclassifications.

Word Clouds for train data:



Figure 1: Unigrams (No pre-processing)



Figure 2: Unigrams (Stopword removal only)



Figure 3: Unigrams (Stemming only)



Figure 4: Unigrams (Stopword removal + stemming)



Figure 5: Unigrams + Bigrams (No pre-processing)



Figure 6: Unigrams + Bigrams (Stopword removal only)



Figure 7: Unigrams + Bigrams (Stemming only)



Figure 8: Unigrams + Bigrams (Stopword removal + stemming)

Word Clouds for test data:



Figure 9: Unigrams (No pre-processing)

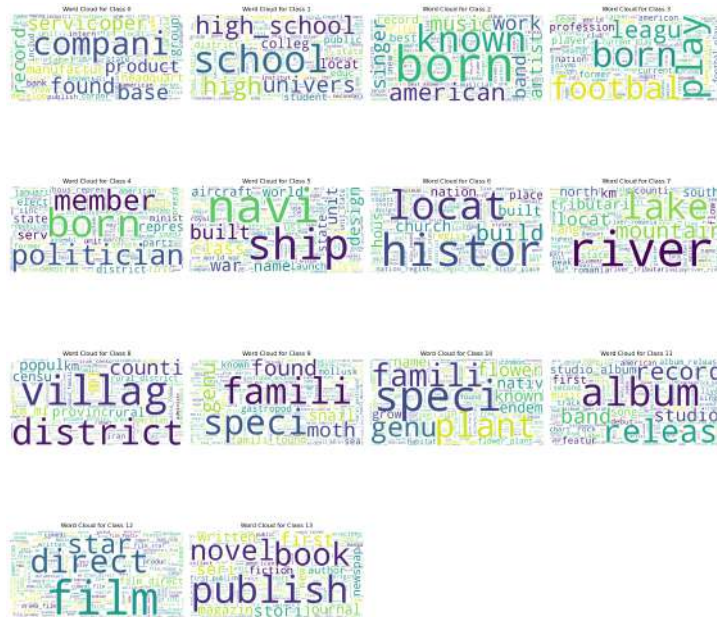


Figure 10: Unigrams (Stopword removal only)



Figure 11: Unigrams (Stemming only)

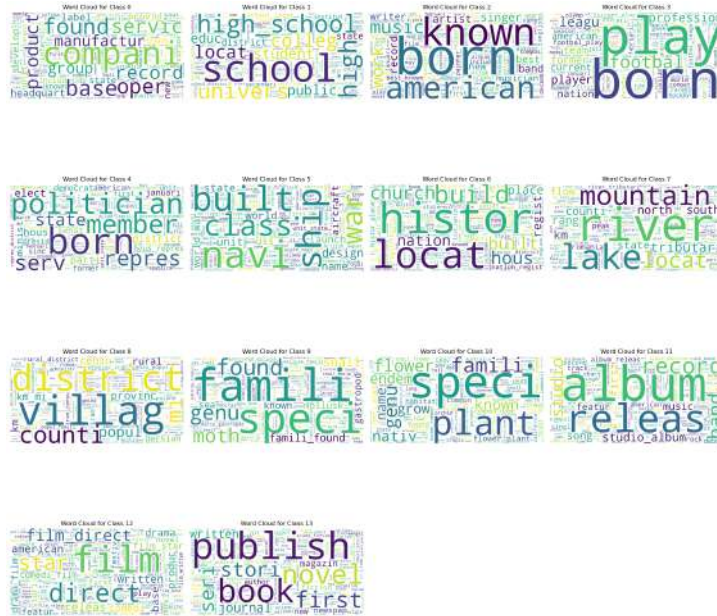


Figure 12: Unigrams (Stopword removal + stemming)



Figure 13: Unigrams + Bigrams (No pre-processing)



Figure 14: Unigrams + Bigrams (Stopword removal only)



Figure 15: Unigrams + Bigrams (Stemming only)

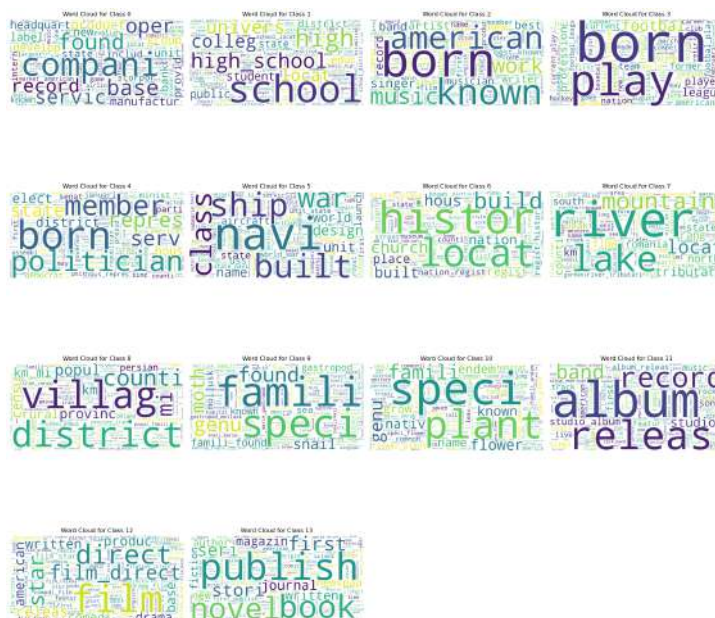


Figure 16: Unigrams + Bigrams (Stopword removal + stemming)

Model comparison using only title as feature

Model	Accuracy	Precision	Recall	F1-score
Unigrams (No pre-processing)	0.95811	0.95846	0.95811	0.95792
Unigrams (Stopword removal only)	0.95500	0.95524	0.95500	0.95469
Unigrams (Stemming only)	0.95363	0.95405	0.95363	0.95341
Unigrams (Stopword removal + stemming)	0.94817	0.94846	0.94817	0.94773
Unigrams + Bigrams (No pre-processing)	0.96737	0.96766	0.96737	0.96727
Unigrams + Bigrams (Stopword removal only)	0.96360	0.96379	0.96360	0.96338
Unigrams + Bigrams (Stemming only)	0.96551	0.96582	0.96551	0.96539
Unigrams + Bigrams (Stopword removal + stemming)	0.95963	0.95992	0.95963	0.95933

Table 3: Comparison on Train Data

Model	Accuracy	Precision	Recall	F1-score
Unigrams (No pre-processing)	0.65080	0.70979	0.65080	0.66065
Unigrams (Stopword removal only)	0.64263	0.70632	0.64263	0.65439
Unigrams (Stemming only)	0.64751	0.70467	0.64751	0.65699
Unigrams (Stopword removal + stemming)	0.63831	0.69936	0.63831	0.64952
Unigrams + Bigrams (No pre-processing)	0.65860	0.71670	0.65860	0.66896
Unigrams + Bigrams (Stopword removal only)	0.64314	0.70650	0.64314	0.65466
Unigrams + Bigrams (Stemming only)	0.65531	0.71170	0.65531	0.65609
Unigrams + Bigrams (Stopword removal + stemming)	0.64006	0.70086	0.64006	0.65104

Table 4: Comparison on Test Data

On using only title as a feature our observations remain the same as when using content. The possible reasons remain the same: loss of context due to stopwords removal/stemming and gain in contextual information due to bigrams inclusion.

Combined model using best results from above

Unigrams + Bigrams without preprocessing work best for both the features (content and title). We concatenate these features and train a naive bayes and we also train a dual parameter naive bayes on the two features individually. The results are summarised in the table below:

Model	Training Accuracy	Test Accuracy
Content Only	0.9900	0.9674
Title Only	0.9553	0.6586
Title + Content (Concatenated)	0.9919	0.9687
Title + Content (Dual Parameter)	0.9917	0.9697

Table 5: Comparison of Model Accuracies

Model with best test accuracy is the Dual Parameter model and it does way better than randomly guessing a class or always guessing the most frequent class.

- **Random Guessing Accuracy:** 0.0714 or 7.14 %
- **Most frequent class in training data:** 0
- **Majority Class Baseline Accuracy:** 0.0714 or 7.14 %

Confusion matrix for our best model:

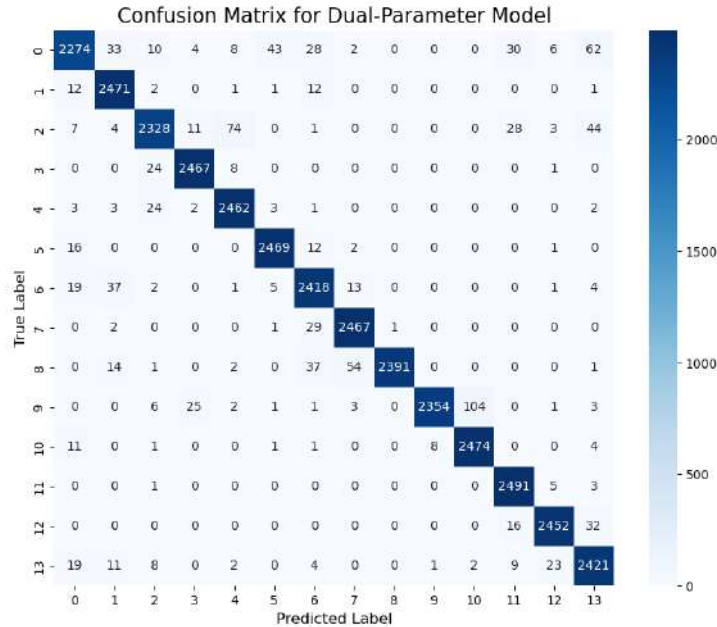


Figure 17: Confusion matrix for Dual Parameter model

The category with the highest value on the diagonal is: Class '11' with 2491 correct predictions. This implies that our model is best at correctly predicting class '11'.

Image Classification using SVM

Mathematics

Primal Problem

Given a dataset of m training examples $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$, where $\mathbf{x}_i \in R^n$ and $y_i \in \{-1, 1\}$, the primal problem for a soft-margin SVM is formulated as a convex optimization problem:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \\ \text{subject to} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, m \\ & \xi_i \geq 0, \quad i = 1, \dots, m \end{aligned}$$

Here, $\mathbf{w}$ is the weight vector, b is the bias, ξ_i are the slack variables for each data point, and $C > 0$ is the regularization parameter.

Lagrangian and Dual Problem

To solve this, we convert to the dual problem using Lagrange multipliers. The Lagrangian is:

$$\mathcal{L}(\mathbf{w}, b, \xi, \alpha, \mu) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m \alpha_i (y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i) - \sum_{i=1}^m \mu_i \xi_i$$

where $\alpha_i \geq 0$ and $\mu_i \geq 0$ are the Lagrange multipliers.

Taking the derivative with respect to the primal variables and setting to zero:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 &\implies \mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i \\ \frac{\partial \mathcal{L}}{\partial b} = 0 &\implies \sum_{i=1}^m \alpha_i y_i = 0 \\ \frac{\partial \mathcal{L}}{\partial \xi_i} = 0 &\implies C - \alpha_i - \mu_i = 0 \implies \alpha_i + \mu_i = C \end{aligned}$$

Substituting these back into the Lagrangian yields the dual objective function:

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

with constraints:

$$\begin{aligned} \sum_{i=1}^m \alpha_i y_i &= 0 \\ 0 &\leq \alpha_i \leq C, \quad i = 1, \dots, m \end{aligned}$$

Kernel Trick

For non-linear separation, we can replace the dot product $\mathbf{x}_i^T \mathbf{x}_j$ with a kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$:

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

with the same constraints.

CVXOPT Standard Form

The 'cvxopt.solvers.qp' function solves quadratic programs in the following standard form:

$$\begin{aligned} \min_{\mathbf{x}} \quad & \frac{1}{2} \mathbf{x}^T P \mathbf{x} + \mathbf{q}^T \mathbf{x} \\ \text{subject to} \quad & G \mathbf{x} \leq \mathbf{h} \\ & A \mathbf{x} = \mathbf{b} \end{aligned}$$

Mapping SVM to CVXOPT format

To map the SVM dual maximization problem to the 'cvxopt' minimization form, we use the following correspondences:

- **Variable:** $\mathbf{x} \equiv \alpha$
- **Objective Function:** We minimize the negative of the dual objective.
 - **Matrix P:** For a kernel K , we define $P_{ij} = y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$.
 - **Vector q:** We set $q_i = -1$ for all $i = 1, \dots, m$.
- **Constraints:**
 - **Equality Constraint** $A \mathbf{x} = \mathbf{b}$: The constraint $\sum_{i=1}^m \alpha_i y_i = 0$ is an equality constraint. We set $A = \mathbf{y}^T$ and $b = 0$.
 - **Inequality Constraint** $G \mathbf{x} \leq \mathbf{h}$: The constraints $0 \leq \alpha_i \leq C$ are two sets of inequalities.

$$\begin{aligned} -\alpha_i &\leq 0 \quad \text{for } i = 1, \dots, m \\ \alpha_i &\leq C \quad \text{for } i = 1, \dots, m \end{aligned}$$

These can be combined into a single matrix inequality where G is a block matrix and $\mathbf{h}$ is a concatenated vector:

$$G = \begin{bmatrix} -I \\ I \end{bmatrix}, \quad \mathbf{h} = \begin{bmatrix} \mathbf{0} \\ C\mathbf{1} \end{bmatrix}$$

Results of our SVM implementation

Linear Kernel

Training completed in **15.95** seconds.

Number of support vectors: **1394**

Percentage of training samples as SVs: **41.81 %**

Test accuracy: **77.18 %**

Calculated w shape: **(3072,)**

Calculated b: **1.2880**

Plot of top 5 support vectors and w vector as images:



Gaussian Kernel

Training completed in **14.90** seconds.

Number of support vectors: **1791**

Percent of training samples as SVs: **53.72%**

Number of matching support vectors with linear kernel: **1137**

Test accuracy: **86.34%**

Note that the test accuracy is higher than that for linear kernel as decision boundary is better captured here.

Plot of top 5 support vectors as images:



Comparison with sklearn inbuilt SVM implementation

Linear Kernel

Training completed in **9.14** seconds.

Number of support vectors: **1378**

Number of matching support vectors with our SVM: **1378**

w diff norm: **0.0173**

b diff: **0.0546**

Test accuracy: **77.18%**

The test accuracy is equal to our implementation using CVXOPT. The w and b are also almost equal. The training time of our implementation is lesser (almost half) compared to the sklearn implementation.

Gaussian Kernel

Training completed in **4.91** seconds.

Number of support vectors: **1777**

Number of matching support vectors with our SVM: **1777**

Test accuracy: **86.49%**

The test accuracy is almost equal (slightly higher) than our implementation with equivalent training times. Moreover, all the support vectors match.

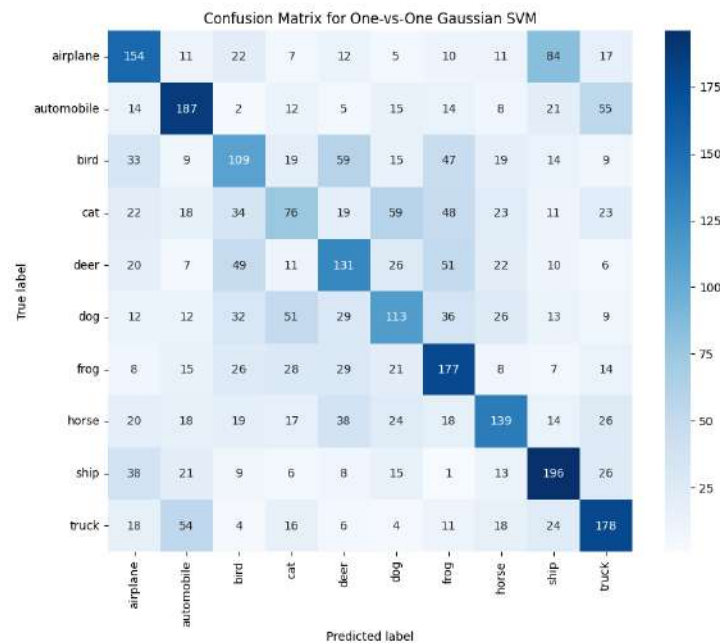
One-vs-One SVM classifier

Our implementation

One-vs-One training completed in **608.80** seconds.

Test accuracy: **43.84%**

Confusion matrix:



Number of misclassified samples: **1870**

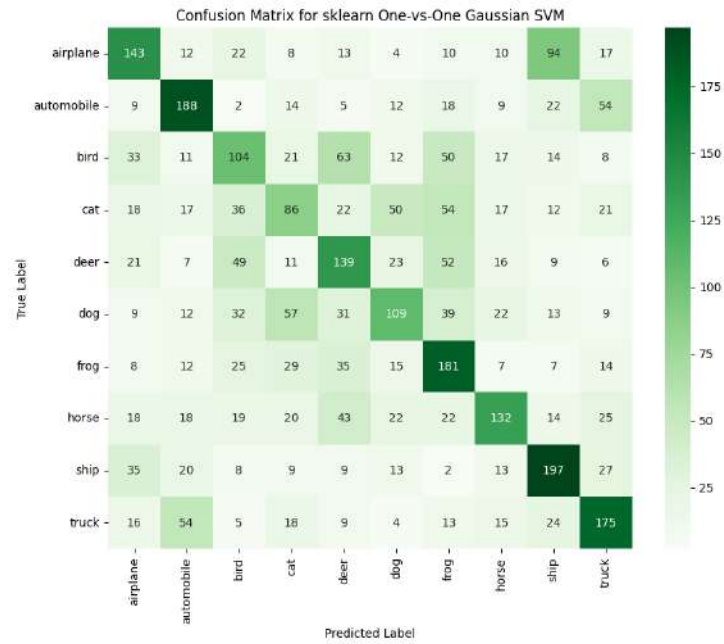


Inbuilt implementation

Training completed in **233.01** seconds.

Test accuracy: **43.66%**

Confusion matrix:



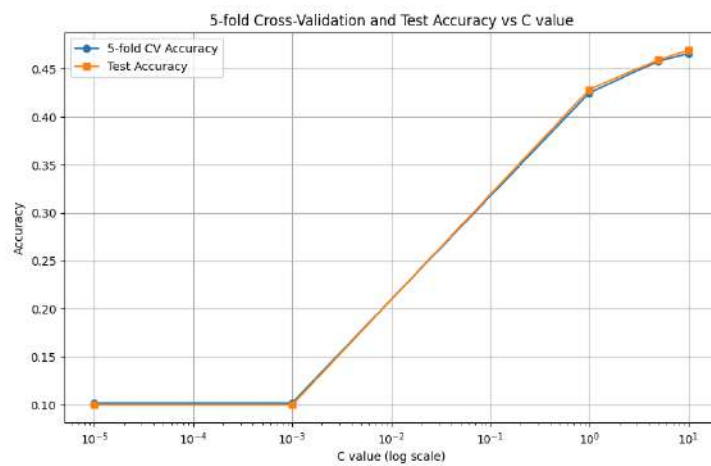
Number of misclassified samples by sklearn: **1876**



Optimal C value and final SVM model based on k-fold cross validation

C Value	5-fold CV Accuracy	Test Accuracy
1e-05	0.1020	0.1000
0.001	0.1020	0.1000
1	0.4250	0.4285
5	0.4581	0.4595
10	0.4660	0.4694

Table 6: Comparison of 5-fold CV and Test Accuracies for different values of C



Best C value based on 5-fold CV accuracy: 10
Final Test Accuracy with C=10: **0.4751**